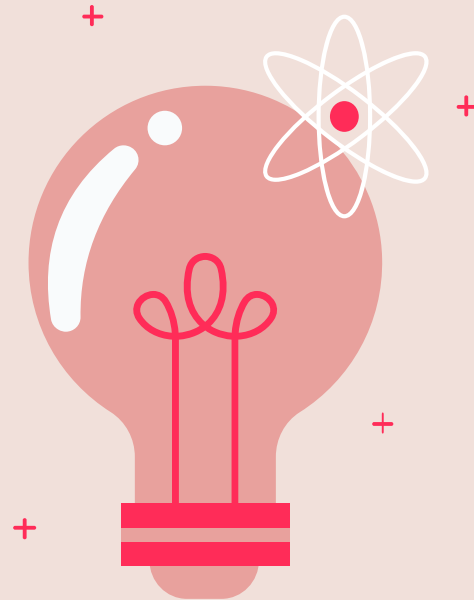


UnMutex

**Voice-piloted Controller
for Mission-Critical Tasks**

Beatrice Calderara

Real Time Embedded Systems @ UniMORE



Perché?

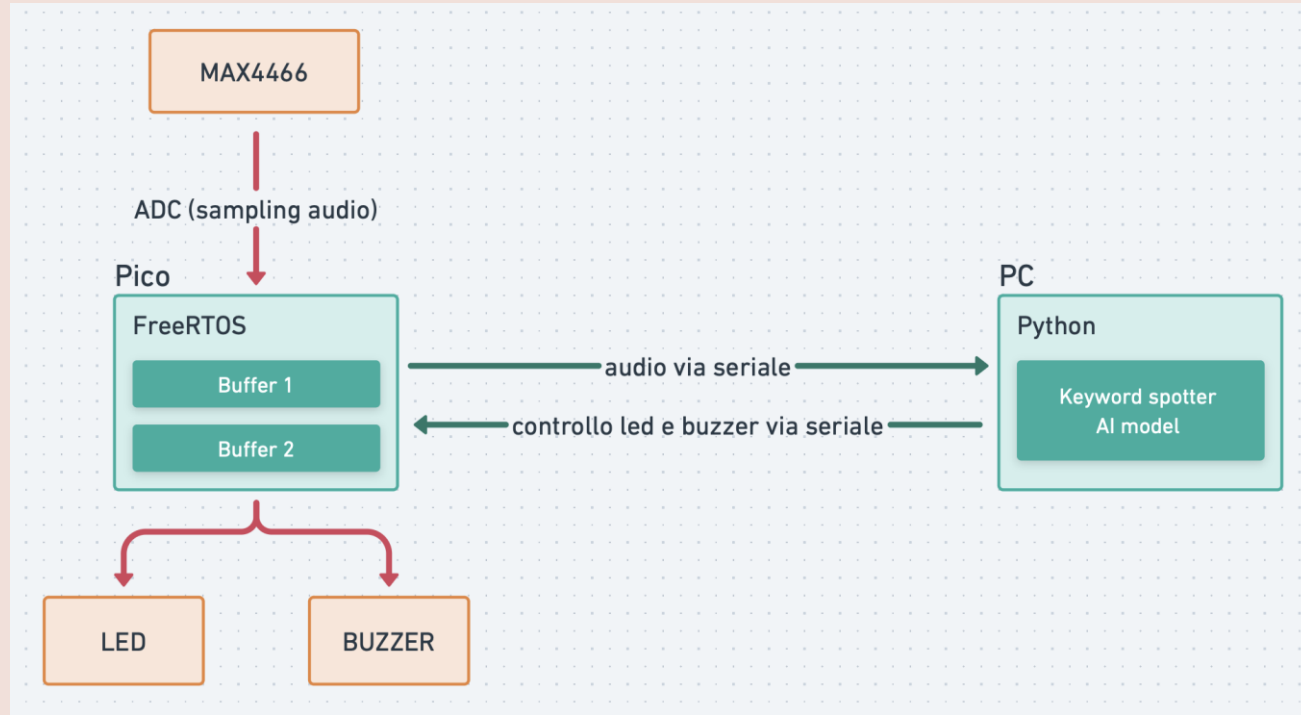
Come integrare efficacemente un sistema embedded in un'applicazione della vita reale, garantendo al contempo controlli affidabili e interazioni naturali?

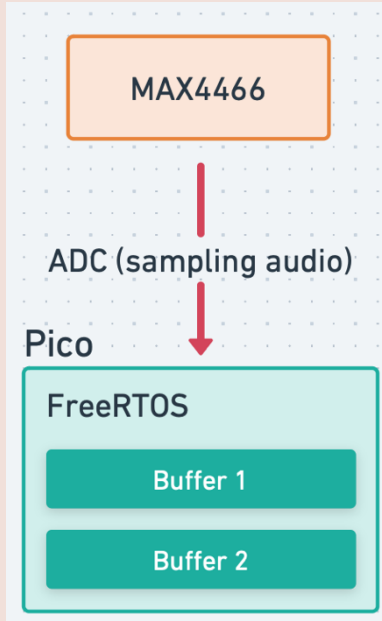
UnMutex

è un controller vocale progettato per gestire task critici in ambienti embedded.

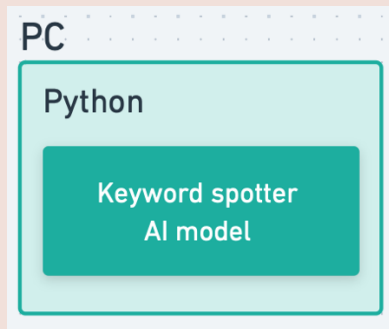
Utilizzando segnali vocali come input, il sistema permette di pilotare dispositivi hardware in modo intuitivo e immediato.

Schema a blocchi dell'architettura del sistema

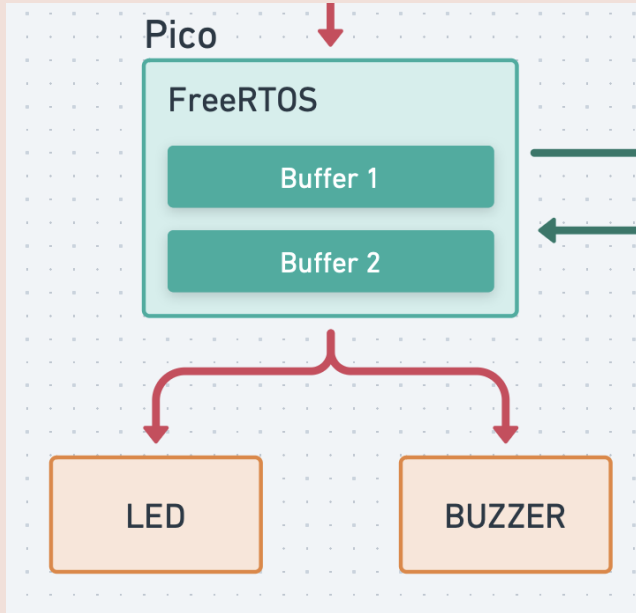




- Il segnale vocale viene acquisito da un **microfono MAX4466**
- L'acquisizione avviene con campionamento a **16 kHz**.
- Il segnale analogico viene digitalizzato dall'ADC a **12 bit** e organizzato in buffer circolari di dimensione fissa (256 sample).
- Quando un buffer è pieno, il relativo puntatore viene messo in coda e un task dedicato invia il contenuto via USB verso il PC, incapsulando i campioni in pacchetti con marker di inizio e fine.



- Sul PC host, un modello di keyword spotting elabora i pacchetti audio in ingresso per riconoscere 4 comandi vocali.
- Il riconoscimento è svolto in tempo reale.
- Ogni parola chiave riconosciuta viene tradotta in un comando seriale, che viene rispedito via USB al Pico.



- Il Pico riceve questi comandi tramite un task di ricezione seriale che effettua il **parsing della stringa** e **notifica i task dedicati al controllo delle uscite**.
- I task per LED e buzzer, sincronizzati tramite mutex per l'accesso esclusivo alle periferiche, implementano le diverse modalità operative.

Modalità operative e comandi vocali

On

Attivazione del LED
onboard mantenendolo
acceso

Off

Spegnimento del LED

Up

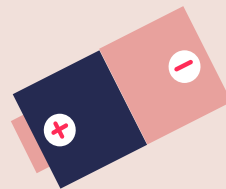
Attivazione del
lampeggio del LED con
frequenza moderata
(10 secondi)

Down

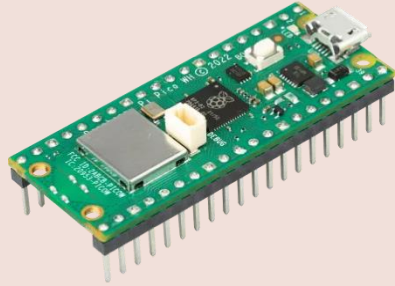
Accensione del buzzer
accompagnata da
lampeggio rapido del LED
(15 secondi)



HARDWARE



Raspberry Pi Pico



MAX4466



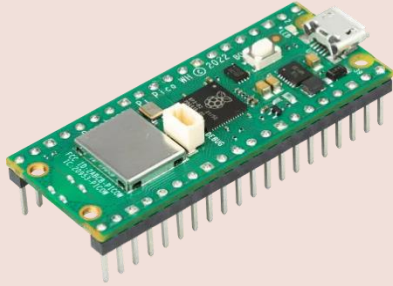
Led



Buzzer



Raspberry Pi Pico



- Processore dual-core ARM Cortex-M0+ @ 133 MHz
- ADC 12-bit per acquisizione audio ad alta fedeltà
- Supporto nativo per FreeRTOS e gestione task real time
- Interfaccia USB integrata per comunicazione seriale

- Microfono a elettrete con amplificatore integrato a basso rumore.
- Dotato di potenziometro per la regolazione del guadagno.
- Opera con tensioni di alimentazione comprese tra 2.4 e 5.5 V
- Fornisce un segnale analogico ad elevata qualità.
- Collegato al pin GP 26 del Raspberry Pi Pico.

MAX4466



Led



- Utilizzato per la segnalazione visuale dello stato del sistema.
- Collegato al pin GPIO 25 del Raspberry Pi Pico (onboard).

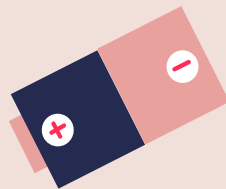
- Utilizzato per la segnalazione audio dello stato del sistema.
- Collegato al pin GPIO 15 del Raspberry Pi Pico.

Buzzer



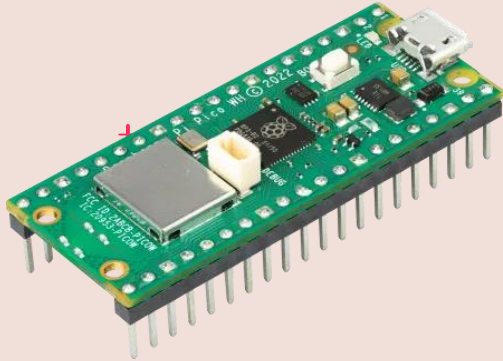


SOFTWARE



Sistema operativo:

FreeRTOS



Linguaggio di
programmazione:

C

PC Host

Linguaggio di
programmazione:

Python



Formato dei pacchetti



Task principali



Gestione della sincronizzazione

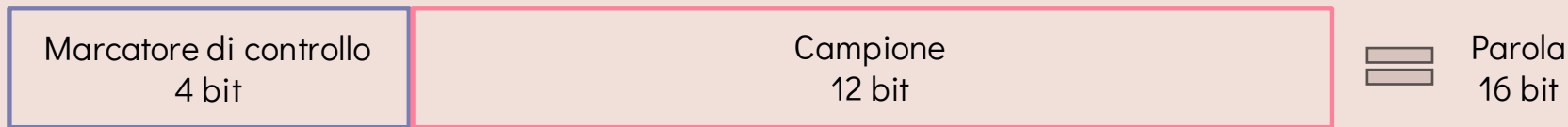


Priorità



Formato dei pacchetti

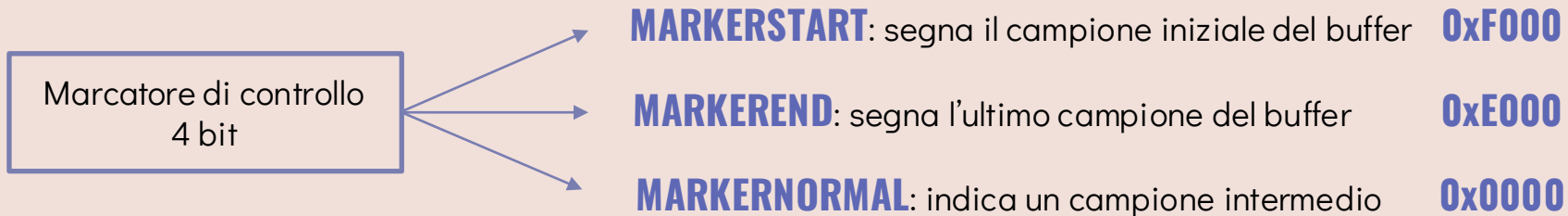
Il segnale audio campionato a 16 kHz è impacchettato in frame contenenti un buffer di 256 campioni.



Il PC riceve un flusso seriale dal quale estrae i pacchetti utilizzando i marcatori come delimitatori. Successivamente, invia indietro al Pico, i comandi di controllo basati sul riconoscimento vocale effettuato.



Formato dei pacchetti



Task di acquisizione audio

Si occupa del **campionamento**, a **16 kHz**, del segnale analogico proveniente dal microfono.

Il sample ADC viene salvato nel buffer attivo e inviato a una *queue* per la trasmissione seriale.



Task principali

Task di acquisizione audio

Si occupa del campionamento, a 16 kHz, del segnale analogico proveniente dal microfono.

Il sample ADC viene salvato nel buffer attivo e inviato a una *queue* per la trasmissione seriale.



Task principali

Task di trasmissione dati audio

Riceve i puntatori ai buffer pieni dalla coda e **invia i dati via USB seriale** al PC.

Task di acquisizione audio

Si occupa del campionamento, a 16 kHz, del segnale analogico proveniente dal microfono.

Il sample ADC viene salvato nel buffer attivo e inviato a una *queue* per la trasmissione seriale.



Task principali

Task di trasmissione dati audio

Riceve i puntatori ai buffer pieni dalla coda e invia i dati via USB seriale al PC.

Task di ricezione comandi seriali

Ascolta i comandi dalla seriale, analizza la stringa ricevuta e notifica il task corrispondente.

Task di acquisizione audio

Si occupa del campionamento, a 16 kHz, del segnale analogico proveniente dal microfono.

Il sample ADC viene salvato nel buffer attivo e inviato a una *queue* per la trasmissione seriale.



Task di trasmissione dati audio

Riceve i puntatori ai buffer pieni dalla coda e invia i dati via USB seriale al PC.

Task di ricezione comandi seriali

Ascolta i comandi dalla seriale, analizza la stringa ricevuta e notifica il task corrispondente.

Task di controllo risorse

Dedicati a gestire il **controllo di LED e buzzer** con sincronizzazione tramite **mutex**, impostati con priorità diverse per rispettare la gerarchia di comando.

Task principali

Mutex

Per la protezione delle risorse hardware condivise.
Evita che più task tentino simultaneamente di modificare lo stato della stessa periferica.



Gestione della sincronizzazione

Code

Per gestire la comunicazione tra il task di acquisizione dati audio e quello di trasmissione seriale.

Paradigma *producer-consumer*: quando un buffer audio viene popolato completamente con nuovi campioni, il *task di acquisizione* inserisce in coda il puntatore a tale buffer; il *task seriale* aspetta di ricevere dalla coda il puntatore e avvia la trasmissione via USB dei dati audio.



Buffer ping-pong

Gerarchia di priorità implementata per assicurare che le operazioni più critiche prevarichino su quelle meno urgenti.

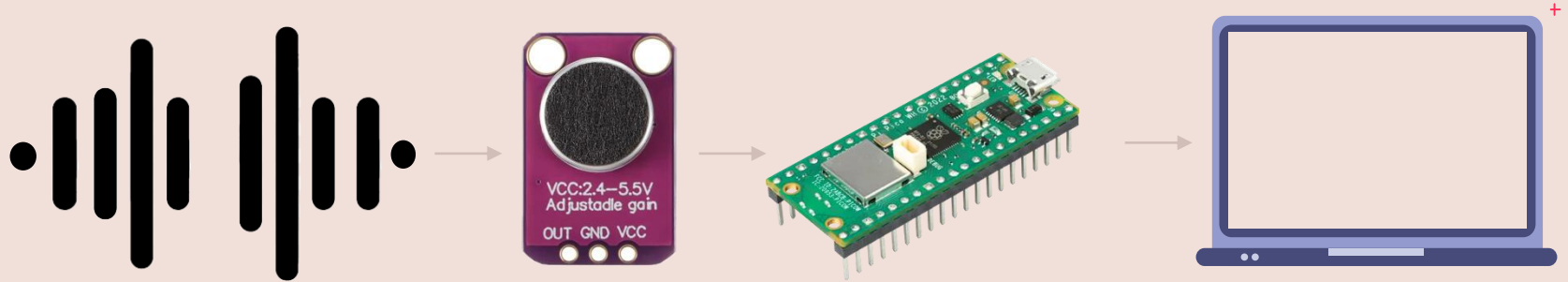
- 5 **vTaskAudioStream**: gestione del campionamento audio, la continuità del flusso dati è fondamentale per la qualità del riconoscimento vocale.
- 4 **vTaskAlarm**: comando down, deve attivare immediatamente buzzer e lampeggio rapido del LED, funzione critica di allarme.
- 3 **vTaskBlink**: comando up, permette il lampeggio del LED senza bloccare task più urgenti.
- 2 **vTaskOff**: comando off, spegnimento del LED
- 1 **vTaskOn**: comando on, accensione del LED
- 1 **vTaskSerialReceiver**: ricezione dei comandi dal PC



Priorità

Modello di Keyword Spotting

Il modello di keyword spotting si occupa di riconoscere specifiche parole chiave all'interno del flusso audio ricevuto dal Raspberry Pi Pico.



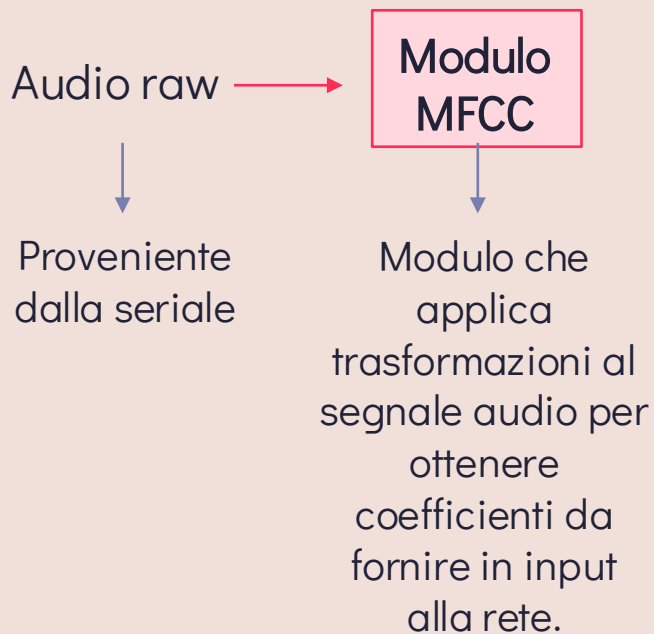
Modello di Keyword Spotting

Audio raw

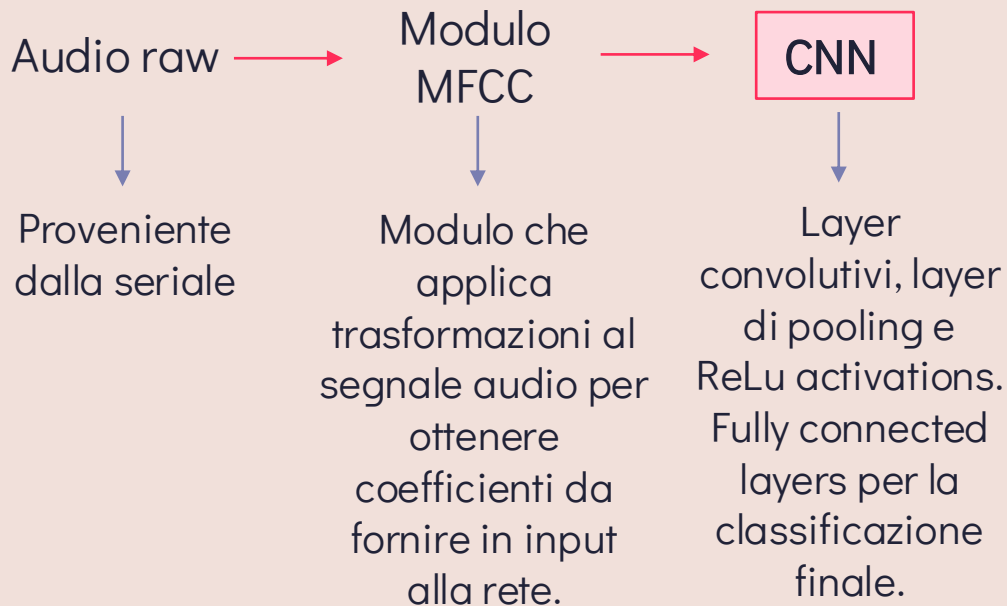


Proveniente
dalla seriale

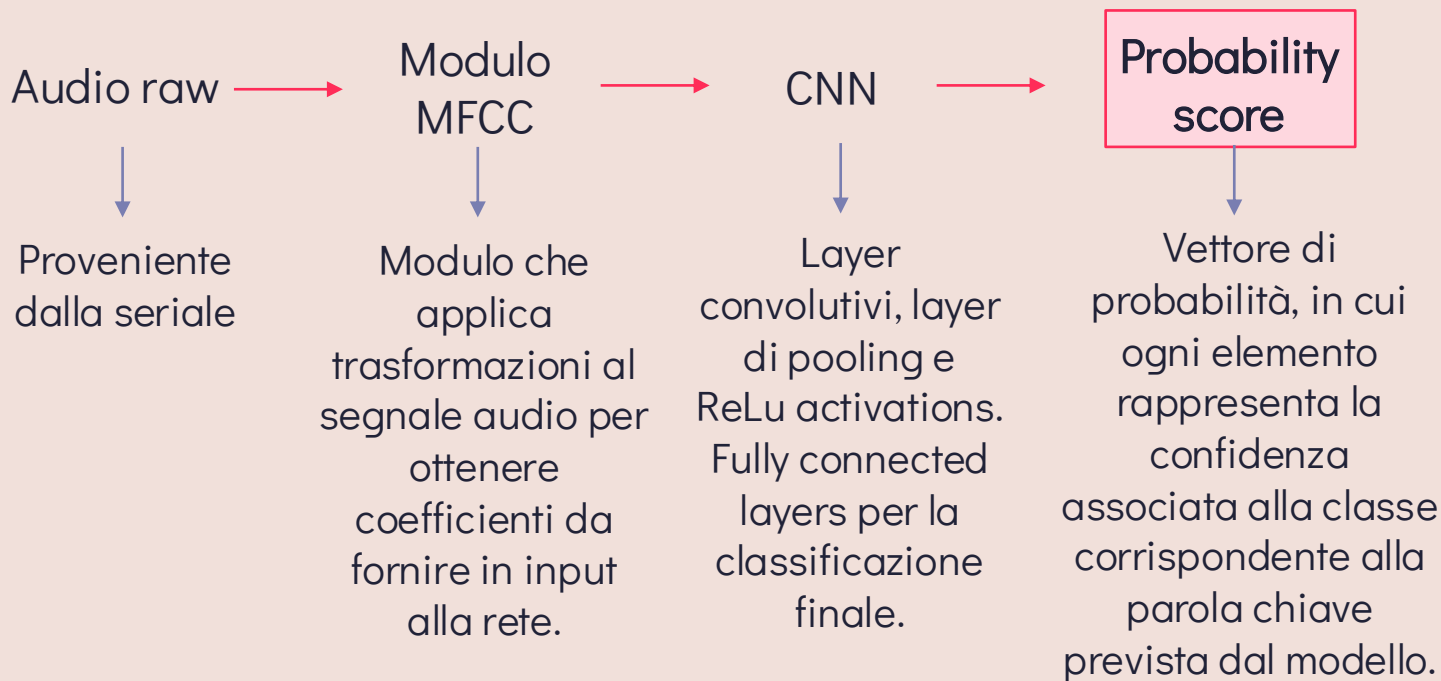
Modello di Keyword Spotting



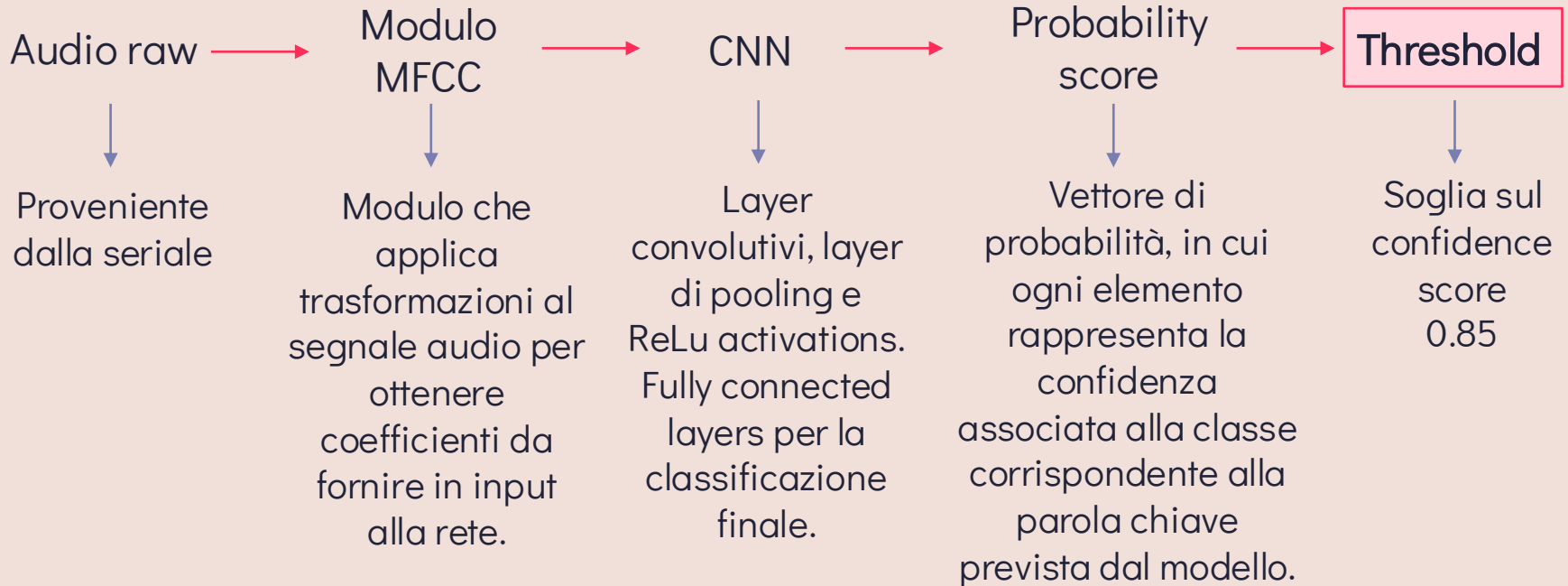
Modello di Keyword Spotting



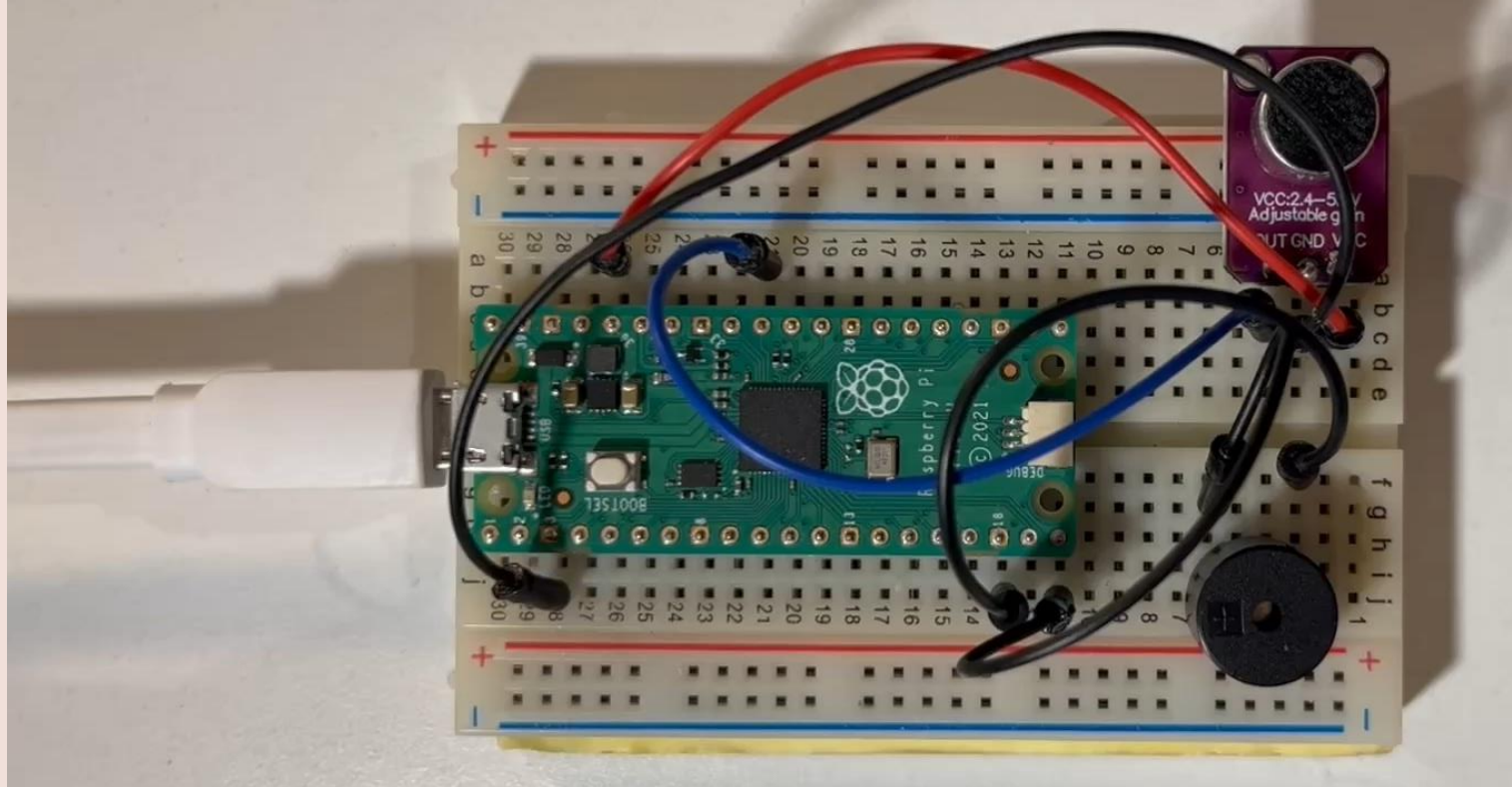
Modello di Keyword Spotting



Modello di Keyword Spotting



Off priorità su On



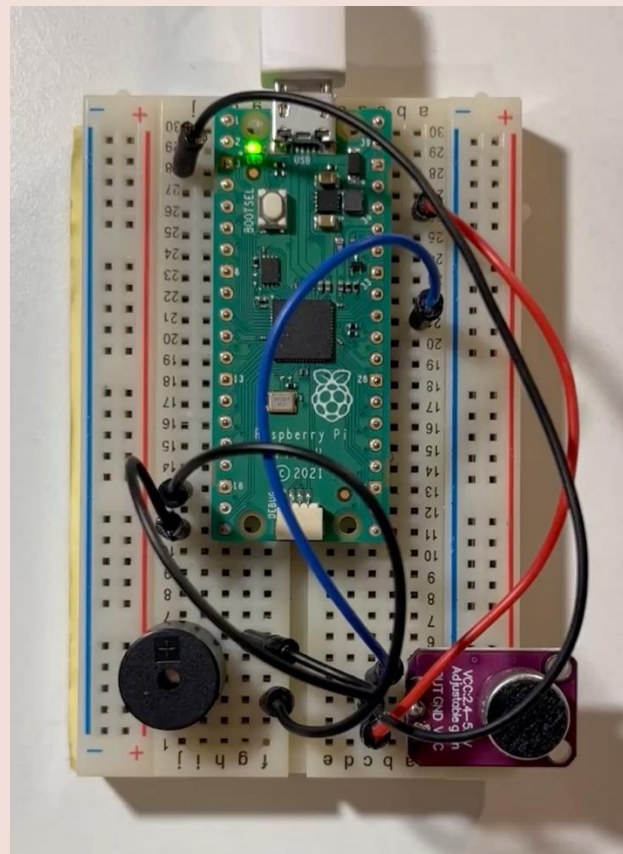
Down priorità su Up, Up priorità su On

```
✓✓✓ Rilevato 'on' con confidenza sufficiente → Invia comando ON  
→ Comando inviato: ON
```

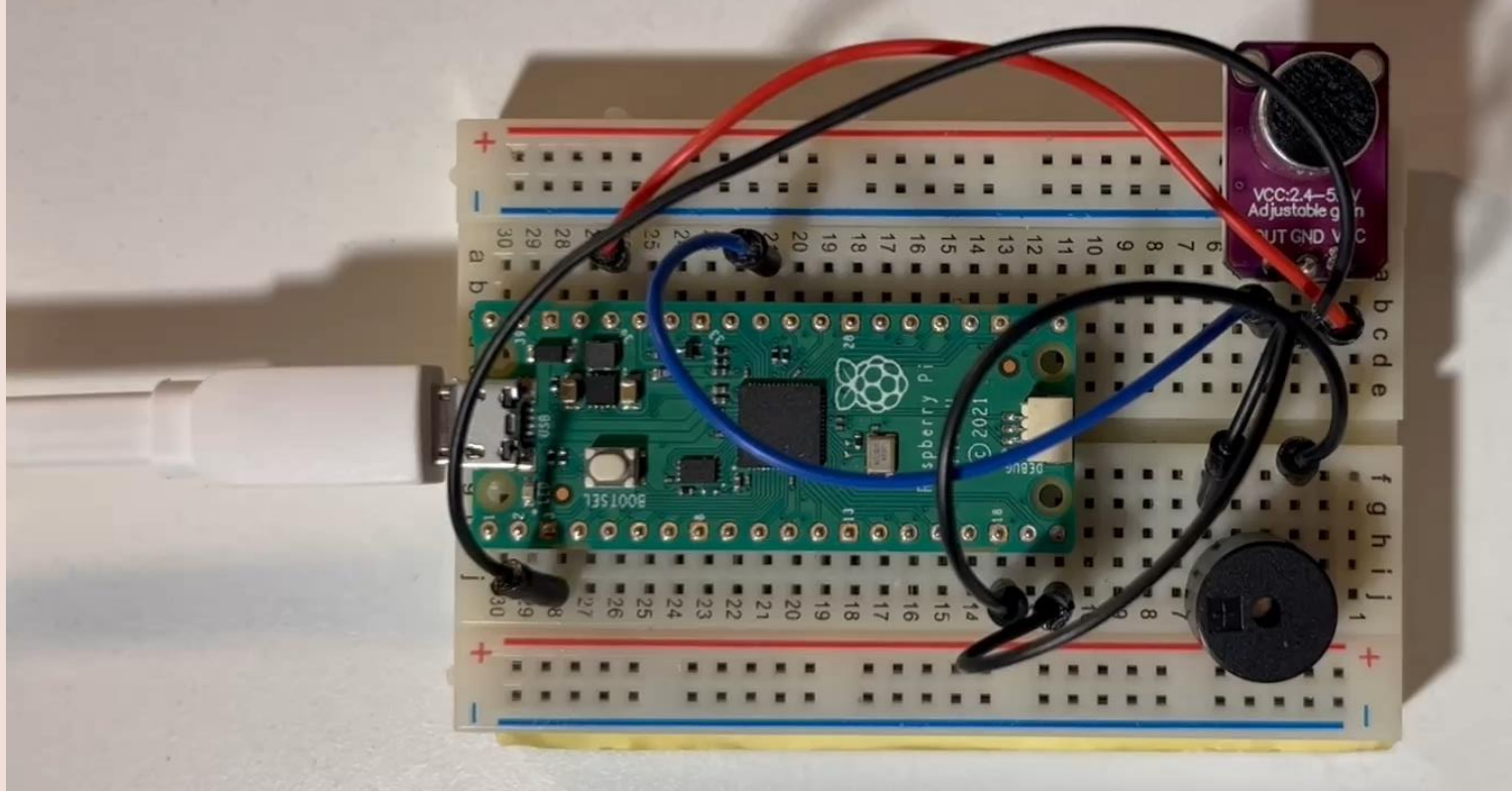
```
✓✓✓ Rilevato 'up' con confidenza sufficiente → Invia comando BLINK  
→ Comando inviato: BLINK
```

```
✓✓✓ Rilevato 'down' con confidenza sufficiente → Invia comando ALARM  
→ Comando inviato: ALARM
```

Il comando blink viene interrotto da Alarm che ha priorità maggiore, e continua la sua esecuzione una volta terminata l'esecuzione del comando Alarm, inoltre Blink NON viene interrotto da On, che ha priorità minore.



Down priorità su On



Conclusione

UnMutex è estensibile e può essere adattato a molteplici contesti e applicazioni reali quali:

- notifica prioritaria di allarmi o emergenze, che devono sovrascrivere altri comandi o task in esecuzione,
- controllo vocale di dispositivi domotici, quali luci, termostati, o serrature, per ambienti domestici o aziendali,
- interfacciamento con macchinari industriali dove la sicurezza personale richiede l'attivazione immediata di procedure di emergenza basate su comandi vocali,
- applicazioni nel settore medico per la gestione hands-free di dispositivi medici o di monitoraggio.

Grazie per l'attenzione



300033@studenti.unimore.it



<https://github.com/beatrice-calderara/RTES-project.git>